

“Chirp” - Krypterad Chatt-app

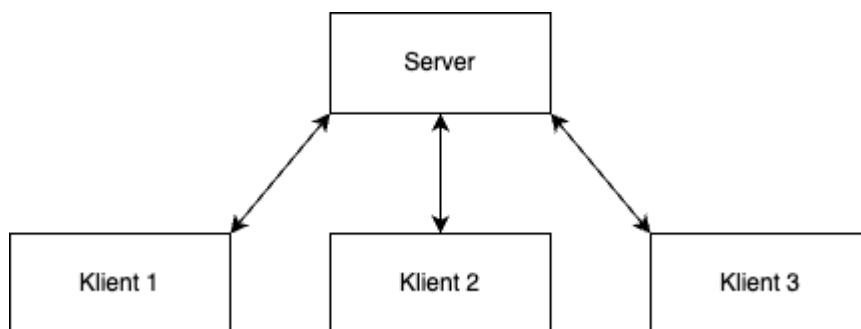
Louis Guerpillon

Chirp är en säker platform där användare får välja ett användarnamn, säkert koppla upp sig till en server, och kommunicera med varandra över ett nätverk. Meddelanden krypteras med algoritmen “AES”, vilket sker innan meddelandena skickas, och dekrypteras när de tas emot. För ytterligare äkerhet signeras även meddelanden digitalt med RSA för att verifiera sändarens identitet. Vid utökning av applikationens funktioner kommer denna del av appen vara lite mer användbar, då man i appens nuläge bara anger ett namn som identitet. Appen hanterar anslutning, kommunikation (msg), och givetvis även leave. Användare ser när andra klienter joinar eller lämnar chatten. **All** kommunikation sker med end-to-end kryptering. Servern som används är i nuläget den lokala localhost , på port 9494. Detta då programmet måste köras gentemot en egen server pga keys.

Framtagande:

1. Idé

Jag har relativt länge velat ha ett sidoprojekt som inkorporerar någon form av chattklient. I exemplen för gesällproven såg jag då beskrivningen “*Chatsystem som använder stark signering/verifiering och eller kryptering/dekryptering med hjälp av klasserna SignedObject och SealedObject*”, vilket jag fastnade på då det lät spännande och nytt. Jag tyckte även om scopet för projektet som var koncis men erbjöd rum för utökning i framtiden. Med tanke på uppgifterna vi fått göra under kursens gång var arkitekturen jag hade i tankarna ganska baserad på denna nya kunskap om klienter och servrar.



Steg 2: Grundläggande utveckling

I denna fas inspirerades jag av uppgift 2.1.1 som introducerade mig till stream sockets på klientsidan.

Jag har ingen kodsnuitt som demonstrerar hur klienten såg ut i ett första skede, dock vet jag att jag först och främst såg till att ha en funktionerande klient lik den som utvecklades under 2.1.1 fast mer standardiserad.

Steg 3: Kryptering:

Efter att ha byggt min klient var det dags att kryptera. Jag använde mig av AES för krypteringen då den tipsades om under uppgift 4.1.1 och ska vara säker samt snabb enligt min research. För detta behövde jag skapa en ny klass som skötte generering samt lagring av så kallade krypteringsnycklar. Det tuffaste här var nog att förstå sig på konceptet och begreppen på en teoretisk och konceptuell nivå vilket krävde en del youtube och online-research. Därefter introducerades enkrypteringselement direkt i chattklienten som såg ut lite som slutliga koden minus signeringen.

```
private String encryptAndSignMessage(String message) throws Exception {
    Signature signature = Signature.getInstance("SHA256withRSA");
    SignedObject signedContent = new SignedObject(message, keyManager.getSigningKeys().getPrivate(), signature);

    ByteArrayOutputStream bos = new ByteArrayOutputStream();
    ObjectOutputStream oos = new ObjectOutputStream(bos);
    oos.writeObject(signedContent);
    oos.close();

    Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
    cipher.init(Cipher.ENCRYPT_MODE, keyManager.getEncryptionKey());
    byte[] encryptedBytes = cipher.doFinal(bos.toByteArray());

    return Base64.getEncoder().encodeToString(encryptedBytes);
}

private String decryptAndVerifyMessage(String encryptedBase64) throws Exception {
    byte[] encryptedBytes = Base64.getDecoder().decode(encryptedBase64);

    Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
    cipher.init(Cipher.DECRYPT_MODE, keyManager.getEncryptionKey());
    byte[] decryptedBytes = cipher.doFinal(encryptedBytes);

    ByteArrayInputStream bis = new ByteArrayInputStream(decryptedBytes);
    ObjectInputStream ois = new ObjectInputStream(bis);
    SignedObject signedObject = (SignedObject) ois.readObject();
    ois.close();

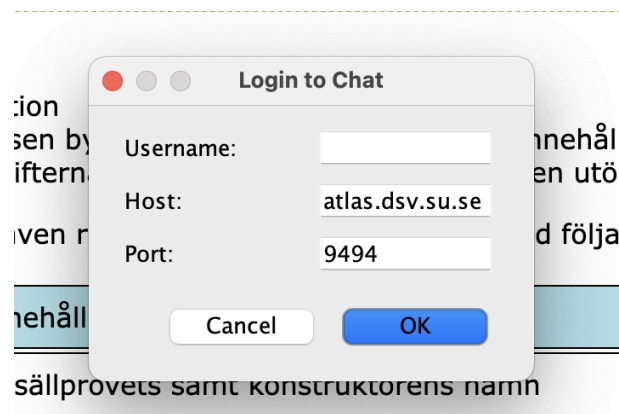
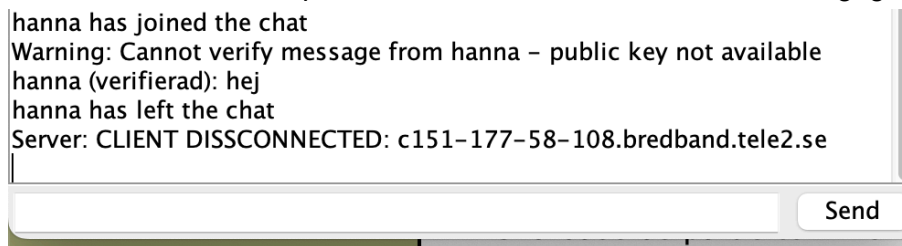
    Signature signature = Signature.getInstance("SHA256withRSA");
    boolean verified = signedObject.verify(keyManager.getSigningKeys().getPublic(), signature);
}
```

Steg 4: Signering

Då uppgiftens initiala "pitch" hade med signering i titeln tänkte jag att signering vore kul att inkorporera med. Detta höjer säkerheten, vilket kändes som en bra utmaning. Digital signering sköttes med RSA och var lite enklare att förstå sig på. encryptMessage samt decryptMessage - funktionerna utökades till "encryptAndSign/decryptAndVerifyMessage()" vilket gjorde flödet ännu mer robust och säkert.

Steg 5: Puts, GUI

I slutstegen provade kände jag att det ändå var dags att ta mig ut ur min CLI-comfort zone och utveckla en väldigt grundläggande UI till appen. Klienten behövde göras om från dess "system.out.print"-baserade CLI funktionalitet, till en busenkel GUI med hjälp av JFrame. Användaren fick nu se på det hela med ett litet UI vilket var väldigt givande för "upplevelsen".



skriv essensen i programmet

Steg 6: Revamp och extra kryptering:

Efter detta steg ville jag fortsätta lägga till krypteringselement och se om jag kunde få det att vara lite extra snyggt. Jag valde att göra krypteringen med AES-GCM då jag fick ögonen för en väldigt [edukativ video inom ämnet](#). Programmet funkar som så att nycklar genereras, krypteras, signeras, dekrypteras, verifieras, och allt detta kräver säkert nyckelutbyte mellan användarna. Numera under enkrypteringsprocessen (metoden):

- Meddelandet signeras med användares privata key
- IV (initialiseringsvektor) genereras slumpmässigt för varje meddelande.
- Kombinerar meddelandet med signaturen
- Krypterar kombinerade datan med AES fast i Galois Counter Mode. Använder mig av en shared secret då det var det närmsta jag kunde komma - tänker att det i längden finns bättre lösningar.

Dekrypteringsprocessen:

- Dekrypterar med AES
- Separerar vad som är meddelande och vad som är signatur
- Verifierar signaturen gentemot sändarens public key.

En mer detaljerad beskrivning av denna process kommer senare.

Steg 7: Server

Efter att ha försökt hitta lite olika sätt att hantera det på, insåg jag att det lättaste sättet att få keys skickat från servern var att göra min egna server. Detta gör hela processen något mer komplicerad än om jag använde mig av DSV:s egna, men gjorde krypteringsarbetet mycket bättre. En ny server skrevs därför som innehöll all form av logik kopplat till keys och signerings/krypteringsprocessen. Nedan ser man lite viktiga delar av denna.

```
case "MSG":
    if (tokenizer.countTokens() >= 2) {
        String sender = tokenizer.nextToken();

        StringBuilder msgContent = new StringBuilder();
        while (tokenizer.hasMoreTokens()) {
            msgContent.append(tokenizer.nextToken());
            if (tokenizer.hasMoreTokens()) {
                msgContent.append(str:" ");
            }
        }

        server.relayMessage(sender, msgContent.toString());
    }
    break;

case "REQUEST_KEYS":
    server.sendAllPublicKeys(this);
    break;

default:
```

Form:

CHIRP är strukturerad kring flera klasser och komponenter som samarbetar för att säkerställa kommunikationen mellan användare. Grundläggande är det uppbyggt som följande:

1. ChatClient, här sker det mesta och all initiering av kryptering, dekryptering, och kommunikation.
2. KeyManager: denna klass genererar och lagrar nycklar för användaren

3. Protocol: Definierar meddelandes format, samt typer (alla meddelanden är inte "literally" meddelanden).
4. Securemessage: hanterar säkra meddelanden med kryptering och signering.

Det finns delar av koden som är mer komplexa (eller snarare, var svårare att få grepp om). Några av de centrala delarna är givetvis Klientens `encryptAndSignMessage()` och `decryptAndVerifyMessage()`. I stort är processen delad i sex delar som jag även kommer visualisera nedan med annoterad kod.

- Signering: med RSA
- Paketering: får med allt i en byte array (dvs längd på meddelandet, meddelandet, längd på signatur, och signaturen).
- Skapande av shared key
- Skapande av initialiseringsvektor.
- Skapande av parameter spec och initialisering av krypteringen
- IV kombineras i byte arrayen.

```
private String encryptAndSignMessage(byte[] messageBytes) throws Exception {
    // signera med användarens privata key
    Signature signature = Signature.getInstance("SHA256withRSA");
    signature.initSign(keyManager.getSigningKeys().getPrivate()); // signering med RSA
    signature.update(messageBytes); // krypteras med privata RSA-nyckeln.
    byte[] signatureBytes = signature.sign();

    // kombinera
    ByteArrayOutputStream bos = new ByteArrayOutputStream();
    DataOutputStream dos = new DataOutputStream(bos);

    dos.writeInt(messageBytes.length);
    dos.write(messageBytes); // Paketering - i stort sett kombinerad av
    dos.writeInt(signatureBytes.length); // signaturen samt meddelandet i en byte array
    dos.write(signatureBytes); // 1. längd på meddelande, 2. meddelandet självt, 3.
    dos.close(); // längd på signatur, 4. signaturen själv

    byte[] combinedData = bos.toByteArray();

    // shared secret mellan klienter
    SecretKeySpec sharedKey = new SecretKeySpec("ThisIsASharedSecretKeyFor256Bits".getBytes(), "AES"); // AES-GCM

    Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding"); // skapar delad nyckel som inte är optimalt
    byte[] iv = new byte[12]; // init-vector gör kryptering unik
    SecureRandom random = new SecureRandom(); // GCM kräver en 12-byte init-vektor vilket görs
    random.nextBytes(iv);

    GCMParameterSpec parameterSpec = new GCMParameterSpec(GCM_TAG_LENGTH * 8, iv);
    cipher.init(Cipher.ENCRYPT_MODE, sharedKey, parameterSpec); // skapar parameter spec (det som gcm
    // "går ut på"). initierar kryptering med
    byte[] encryptedData = cipher.doFinal(combinedData); // cipher.init

    ByteArrayOutputStream encryptedOutput = new ByteArrayOutputStream();
    encryptedOutput.write(iv); // viktigt: vektorn läggs till direkt i byte arrayen
    encryptedOutput.write(encryptedData); // som outputas för att kunna dekrypteras

    return Base64.getEncoder().encodeToString(encryptedOutput.toByteArray());
}
```

Vid dekryptering är processen liknande:

1. Depkryptering av data med AES
2. Separera själva meddelandet och signaturen
3. Verifierar så att signaturen stämmer med avsändarens publika key.

Nyckelgenerering vid uppstart gör även signeringen möjlig.

```
public KeyManager() throws NoSuchAlgorithmException {  
    KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");  
    keyGen.initialize(keysize:2048);  
    signingKeys = keyGen.generateKeyPair();  
  
    KeyGenerator keyGenerator = KeyGenerator.getInstance("AES");  
    keyGenerator.init(256);  
    encryptionKey = keyGenerator.generateKey();  
}
```

Nedan finner vi **pseudokod** på hur programmet fungerar, i dess kritiska delar.

ChatClient(host, port, username)

keyManager() = generera våra RSA- och AES-nycklar

anslut till server via socket på host:port

registrera användarnamn och publik nyckel med servern

starta separat tråd för att ta emot meddelanden

returnera klient-objekt

sendChatMessage(messageText)

skapa Protocol.Message-objekt med meddelandetext

serialisera meddelandet till byte-array

skicka byte array till encryptAndSignMessage()

skicka "MSG username encrypted-message" till servern

encryptAndSignMessage()

skapa signatur enligt RSA SHA256

paketera signaturen, dess längd, meddelandet, och meddelandets längd i en och samma bytearray

få den deleade secret keyen (hårdkodad)

initialiseringsvektor skapas för att göra varje krypering unik.

skapar parametrar som behövs av ciphern i GCM

initialisera krypering

skapa bytearray med slutliga krypterade datat

sätt samman vektorn och datat i en och samma output.

decryptAndVerifyMessage()

Dekodera Base64-sträng till byte-array

Extrahera initialiseringsvektorn (IV) från första 12 byten (vi vet ju att den är 12 och lades i början)

Extrahera den krypterade datan från resten av bytena

Hämta den delade hemliga nyckeln igen

Skapa parametrar för AES GCM-dekryptering

Initiera dekryptering

Dekryptera datan

Läs den dekrypterade datan:

Läs längden på meddelandet

Läs själva meddelandet

Läs längden på signaturen

Läs själva signaturen

Hämta avsändarens publika nyckel

Om publika nyckeln finns:

Skapa verifieringsobjekt för RSA-SHA256

Verifiera signaturen med meddelandet

Om signaturen inte stämmer → kasta säkerhetsfel

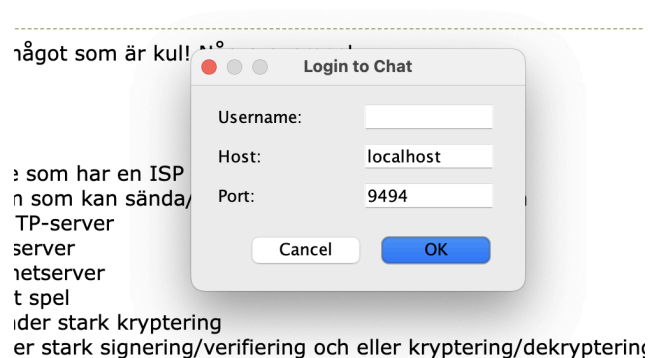
Om ingen publik nyckel finns:

varning

Returnera det dekrypterade meddelandet

Funktion

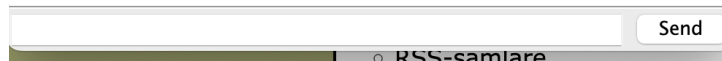
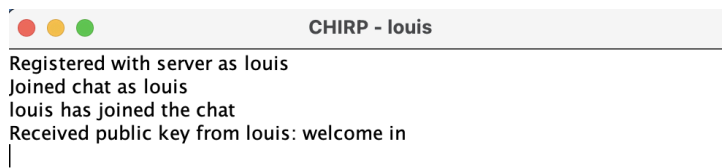
ot exempel på denna pseudokod eftersom vi inte vill att AI sk



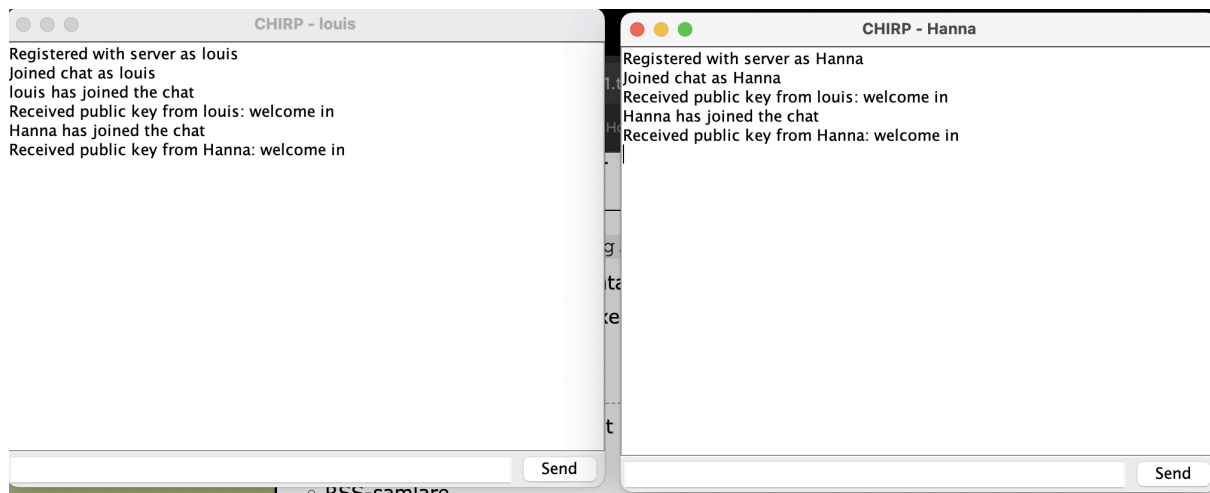
ja ett program som simulerar en webbläsare på en certifierad

Det färdiga programmet välkomnar användaren med tre inputs där denne får välja port, host, och användarnamn. Vid fungerande inmatning kommer användaren in till ett chattrum. Host och port är satt till localhost respektive 9494 som standard men man får byta om man vill, så länge serverns kod är anpassad för denna.

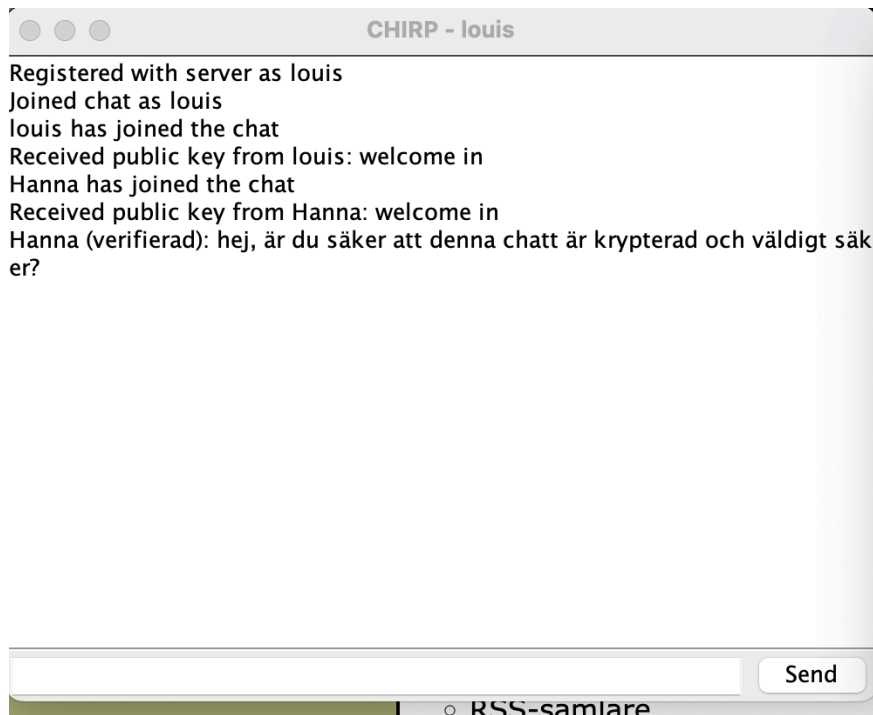
Servern måste vara igång innan man connectar. Väl igång med t:ex “java ChatServer”
kommunicerar den via sin CLI och ger grundläggande information om vad som sker, samt
kommunicerar vilken port den startats på (inspirerat av **2.1.2**).



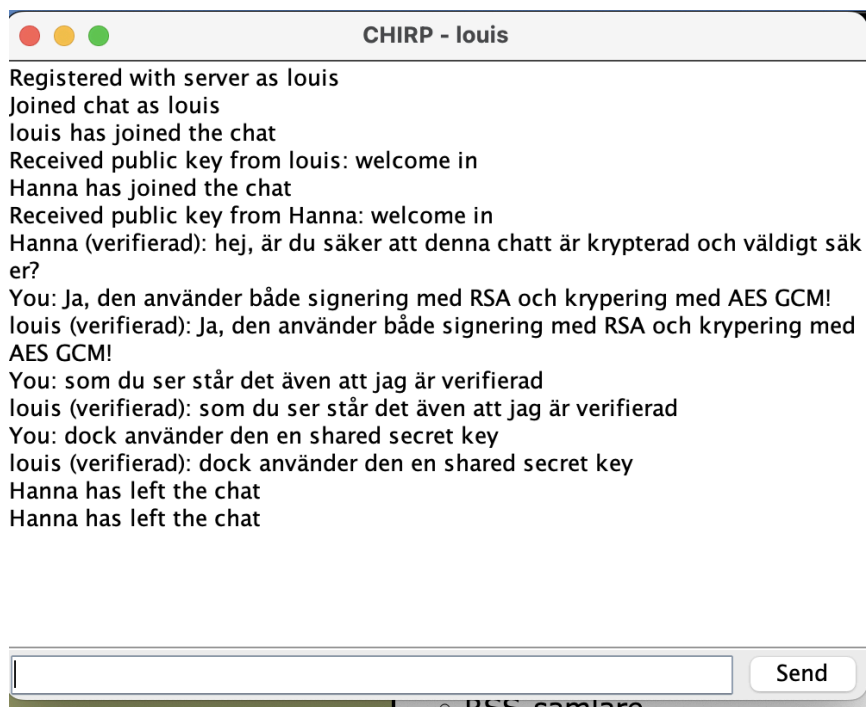
Väl inne i programmet blir du välkommen när servern fått din public key. Nu kan du börja chatta. När andra klienter kopplar upp sig blir du notifierad.



När man får ett meddelande är användarnamnet följt med verifierad ENBART om signaturen godkänns, annars är den markerad som ej verifierad.



När en annan person lämnar chatten, notifieras alla klienter om vem som lämnat.



Vid nedstängning får man gärna även stänga ner servern!